

学号：202300130183	姓名：宋浩宇
实验名称：事件抽取基础实验——文本事件结构分析	
1. 实验目的	
1. 理解事件抽取（Event Extraction）的基本概念。 2. 掌握事件的基本组成（主体、动作、客体、时间等）。 3. 学习从文本中提取事件结构。 4. 理解事件抽取与实体识别、关系抽取的关系。 5. 分析不同句式下事件表达的差异。	
2. 实验原理	
事件抽取是从文本中识别“发生了什么”的过程，其目标是将自然语言转换为结构化表示。 例如： 华为发布了新产品。 可以表示为： 事件：发布 主体：华为 客体：新产品 再如： 2025 年 3 月 15 日，王老师在上海开会。 可以表示为： 事件：开会 主体：王老师 时间：2025 年 3 月 15 日 地点：上海 事件抽取通常建立在： NER + 关系抽取 的基础上	
3. 实验步骤与代码	
实验步骤： 1. 输入实验文本 2. 对句子进行分词或简单句法分析 3. 找出事件触发词（如“发布”、“创立”、“开会”） 4. 识别事件主体（通常为主语） 5. 识别事件对象（宾语） 6. 提取时间、地点信息（如有） 7. 构造事件结构，例如： {主体, 事件, 客体, 时间, 地点} 1. 输出结果并整理	
代码： <pre>from __future__ import annotations import re</pre>	

```

from pathlib import Path
DATA_PATH = r"./exp-7.2-data.txt"
def init_jieba() -> bool:
    """在部分 Python 版本组合下 jieba 可能因 pkg_resources 等问题
    无法导入, 此时仍可进行规则抽取."""
    try:
        import jieba
        jieba.initialize()
        return True
    except Exception:
        return False
JIEBA_OK = init_jieba()
def parse_data_file(path: str) -> tuple[list[str], str]:
    events: list[str] = []
    integrated = ""
    mode = None
    for raw in
Path(path).read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line:
            continue
        if line == "[事件句]":
            mode = "evt"
            continue
        if line == "[综合抽取]":
            mode = "int"
            continue
        if mode == "evt":
            events.append(line)
        elif mode == "int":
            integrated = line
    return events, integrated
def tokenize_hint(sentence: str) -> str:
    if not JIEBA_OK:
        return "(jieba 不可用, 已跳过分词参考。任务书环境为
Python3.8+, 一般可正常 pip install jieba。)"
    import jieba.posseg as pseg
    pairs = list(pseg.cut(sentence))
    return " ".join(f"{w}/{f}" for w, f in pairs)
def strip_punct_end(s: str) -> str:
    return s.strip().rstrip("。！？")
def extract_time_prefix(sentence: str) -> tuple[str, str]:
    m = re.match(r"^(\d{4}\s*年\s*\d{1,2}\s*月\s*\d{1,2}\s*
日)\s*[.,]\s*(.+)$", sentence.strip())

```

```

if not m:
    return "", sentence.strip()
t = re.sub(r"\s+", "", m.group(1))
rest = m.group(2).strip()
return t, rest
def extract_event_structure(sentence: str) -> dict:
    """
    返回字典字段与任务书「主体、事件、客体、时间、地点」对齐。
    未抽到的键对应空字符串，便于统一制表打印。
    """
    s0 = strip_punct_end(sentence)
    time, core = extract_time_prefix(s0)
    if not time:
        core = s0
    subj = obj = loc = ""
    trigger = ""
    note = ""
    m = re.match(r"^(.+?)发布了(.+)$", core)
    if m:
        subj, obj = m.group(1).strip(), m.group(2).strip()
        trigger = "发布"
        return _result(time, subj, trigger, obj, loc, note)
    m = re.match(r"^(.+?)创立了(.+)$", core)
    if m:
        subj, obj = m.group(1).strip(), m.group(2).strip()
        trigger = "创立"
        return _result(time, subj, trigger, obj, loc, note)
    m = re.match(r"^(.+?)在(.+?)参加(?:了)?(.+)$", core)
    if m:
        subj, loc, obj = m.group(1).strip(), m.group(2).strip(),
m.group(3).strip()
        trigger = "参加"
        return _result(time, subj, trigger, obj, loc, note)
    m = re.match(r"^(.+?)会在(.+?)开会$", core)
    if m:
        subj, loc = m.group(1).strip(), m.group(2).strip()
        trigger = "开会"
        return _result(time, subj, trigger, obj, loc, note)
    m = re.match(r"^(.+?)买了(.+)$", core)
    if m:
        subj, obj = m.group(1).strip(), m.group(2).strip()
        trigger = "买"
        return _result(time, subj, trigger, obj, loc, note)
    note = "未命中规则模板，请在本机用更复杂句法或深度学习模型扩展。

```

```

    return _result(time, subj, trigger, obj, loc, note)
def _result(time: str, subj: str, trigger: str, obj: str, loc: str,
note: str) -> dict:
    return {
        "时间": time,
        "主体": subj,
        "事件触发词": trigger,
        "客体": obj,
        "地点": loc,
        "备注": note,
    }
def is_complete(ev: dict) -> str:
    need = ["主体", "事件触发词"]
    ok = all(ev.get(k) for k in need)
    if ok and ev.get("事件触发词") in ("发布", "创立", "买"):
        ok = ok and bool(ev.get("客体"))
    if ok and ev.get("事件触发词") == "参加":
        ok = ok and bool(ev.get("客体")) and bool(ev.get("地点"))
    if ok and ev.get("事件触发词") == "开会":
        ok = ok and bool(ev.get("地点"))
    return "结构较完整" if ok else "结构不完整(缺少关键槽位)"
def integrated_ie(text: str) -> None:
    """任务书「综合实验」示例输出。"""
    t = strip_punct_end(text)
    print("\n--- 综合信息抽取(任务书给定输入) ---")
    print(f"输入: {text}")
    m = re.match(r"^(.+?) 在(.+?) 讲授(.+)$", t)
    if not m:
        print("(综合示例未命中规则, 请检查数据文件中的句子是否与任
务书一致。)")
        return
    who, org, course = m.group(1).strip(), m.group(2).strip(),
m.group(3).strip()
    print("\n 实体(规则标注, 便于与实验十三模型结果对照):")
    print(f"   {who}(人)")
    print(f"   {org}(机构)")
    print(f"   {course}(课程)")
    print("\n 关系(模板化输出, 对应任务书示例):")
    print(f"   ({who}, 任教于, {org})")
    print("\n 事件(结构化字典):")
    ev = {
        "主体": who,
        "行为": "讲授",

```

```

        "客体": course,
        "地点": org,
    }
    print(f"    {ev}")
    print("\njieba 分词与词性参考:")
    print(f"    {tokenize_hint(t)}")
def print_event_block(title: str, sentences: list[str]) -> None:
    print(f"\n--- {title} ---")
    print("序号 | 原句 | 时间 | 主体 | 触发词 | 客体 | 地点 | 完整性提示 | 备注")
    print("-" * 96)
    for i, s in enumerate(sentences, 1):
        ev = extract_event_structure(s)
        print(
            f"{i:>2} | {s} | {ev['时间'] or '(无)'} | {ev['主体'] or '(无)'} | "
            f"{ev['事件触发词'] or '(无)'} | {ev['客体'] or '(无)'} | {ev['地点'] or '(无)'} | "
            f"{is_complete(ev)} | {ev['备注']}"
        )
        print(f"    分词参考: {tokenize_hint(s)}")
def main():
    print(f"数据文件: {DATA_PATH}")
    print(f"jieba 分词参考: {'已启用' if JIEBA_OK else '未启用(规则抽取不受影响)'}")
    events, integrated = parse_data_file(DATA_PATH)
    print("\n--- 实验十四 事件抽取:逐句结构化结果(便于报告截图) ---")
    print_event_block("事件句(任务书第五节数据)", events)
    if integrated:
        integrated_ie(integrated)
    else:
        print("\n(数据文件中未找到 [综合抽取] 段落, 已跳过综合示例。)")
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

```

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业\第七周实验\实验十四-事件抽取实验——结构化语义理解.py
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\23676\AppData\Local\Temp\jieba.cache
Loading model cost 0.536 seconds.

```

Prefix dict has been built successfully.

数据文件: ./exp-7.2-data.txt

jieba 分词参考: 已启用

--- 实验十四 事件抽取: 逐句结构化结果 (便于报告截图) ---

--- 事件句 (任务书第五节数据) ---

序号 | 原句 | 时间 | 主体 | 触发词 | 客体 | 地点 | 完整性提示 | 备注

- 1 | 华为发布了新产品。 | (无) | 华为 | 发布 | 新产品 | (无) | 结构较完整 |
分词参考: 华为/nr 发布/v 了/ul 新/a 产品/n 。/x
- 2 | 马云创立了阿里巴巴。 | (无) | 马云 | 创立 | 阿里巴巴 | (无) | 结构较完整 |
分词参考: 马云/nr 创立/v 了/ul 阿里巴巴/nrt 。/x
- 3 | 张三在北京参加会议。 | (无) | 张三 | 参加 | 会议 | 北京 | 结构较完整 |
分词参考: 张三/nr 在/p 北京/ns 参加/v 会议/n 。/x
- 4 | 2025 年 3 月 15 日, 王老师在上海开会。 | 2025 年 3 月 15 日 | 王老师 | 开会 | (无) | 上海 | 结构较完整 |
分词参考: 2025/m 年/m 3/m 月/m 15/m 日/m , /x 王老师/nr 在/p 上海/ns 开会/v 。/x
- 5 | 小明买了一本书。 | (无) | 小明 | 买 | 一本书 | (无) | 结构较完整 |
分词参考: 小明/nr 买/v 了/ul 一/m 本书/r 。/x

--- 综合信息抽取 (任务书给定输入) ---

输入: 王老师在清华大学讲授人工智能课程。

实体 (规则标注, 便于与实验十三模型结果对照):

王老师 (人)
清华大学 (机构)
人工智能课程 (课程)

关系 (模板化输出, 对应任务书示例):

(王老师, 任教于, 清华大学)

事件 (结构化字典):

{'主体': '王老师', '行为': '讲授', '客体': '人工智能课程', '地点': '清华大学'}

jieba 分词与词性参考:

王老师/nr 在/p 清华大学/nt 讲授/n 人工智能/n 课程/n

进程已结束, 退出代码为 0

图表结果为:

无图表结果

5. 结果分析

1.每个句子抽取出了哪些事件信息？

答：第 1 句，时间无，主体华为，触发词发布，客体新产品，地点无。第 2 句，时间无，主体马云，触发词创立，客体阿里巴巴，地点无。第 3 句，时间无，主体张三，触发词参加，客体会议，地点北京。第 4 句，时间 2025 年 3 月 15 日，主体王老师，触发词开会，客体无，地点上海。第 5 句，时间无，主体小明，触发词买，客体一本书，地点无。综合示例句另见第 6 题。

2.哪些事件结构完整？哪些不完整？

答：程序对五句事件句的完整性提示均为“结构较完整”，这与本实验采用的规则模板有关，只要关键槽位按规则匹配成功即判为较完整。若从语义槽位角度细抠，第 4 句“开会”类事件在输出里客体为空，若任务书要求事件必须带明确受事或议题，可视为相对不完整，可在报告中作为讨论点写出。其余几句在主体、触发词、客体或地点上覆盖了任务书示例关心的主要要素。

3.不同句式是否影响事件抽取效果？

答：会影响。本程序主要依赖固定正则与少量句式模板，例如“谁发布了什么”“谁在何地参加什么”“谁在何地开会”“谁买了什么”等。句式一旦偏离模板，例如把关键动词换成长距离搭配、插入大量从句或省略主语，规则抽取更容易漏槽或匹配到错误边界。本次五句均为短句且与模板高度一致，因此输出稳定，但这恰恰说明句式变化会改变可抽取性。

4.事件抽取相比关系抽取增加了哪些信息？

答：关系抽取通常强调实体对之间的谓词关系，例如谁与谁存在某种联系。事件抽取在此基础上更强调“发生了什么”，往往包含触发词或事件类型、与事件绑定的论元角色，例如时间、地点、客体等，并把这些要素组织成同一次事件的结构化表示。综合示例里除关系三元组外，还给出带主体、行为、客体、地点的事件字典，这就是在关系之外补充的过程性与情境信息。

5.哪些句子较难进行事件抽取？为什么？

答：就本次运行而言，五句均命中规则，未出现备注里的“未命中规则模板”。若推广到更难文本，通常包括以下情况，一是主语省略或代指链长，主体难以稳定绑定，二是同一动词承担多种义项，触发词与事件类型歧义，三是时间地点以隐含方式出现，不落在正则可覆盖的表面形式上，四是并列、转折导致一句多事件，需要切分与对齐。写报告时可把第 4 句作为“时间前置、逗号后另起小句”的略复杂句式样例，说明规则系统对分隔与时间前缀的依赖。

6.综合实验：构建一个简单信息抽取系统

输入文本：

王老师在清华大学讲授人工智能课程。

输出：
实体：
王老师（人）
清华大学（机构）
人工智能课程（课程）
关系：
(王老师,任教于,清华大学)
事件：
{主体:王老师,行为:讲授,客体:人工智能课程,地点:清华大学}
说明：上述“事件”一行严格按当次控制台打印的字典整理，键名与取值与程序输出一致。

6. 实验总结

本次在启用 jieba 的前提下，运行实验十四脚本，对 exp-7.2-data.txt 中五条事件句按规则模板输出时间、主体、触发词、客体、地点及完整性提示，并附分词词性行。综合示例句按同一脚本输出实体列表、任教关系三元组以及带主体、行为、客体、地点的事件字典。程序退出码为 0，表格与综合块与任务书结构一致，可直接用于报告整理。
使用 LLM 辅助完成代码编写